CIFAR-100 Image Classification with a Designed Convolutional Network

Abstract

This report presents a two-part deep-learning project on CIFAR-100. Part A studies a complete image-classification pipeline, from a deliberately simple convolutional baseline to a custom wide residual network. Accuracy is improved step by step by stabilizing optimization, adding normalization and a warmup–cosine schedule, redesigning the spatial hierarchy, increasing capacity, and training with augmentation. The main conclusions are that augmentation is budget-dependent and that width plus residual depth is more reliable than depth alone. After retraining on all 50,000 training images, a three-model ensemble with horizontal-flip test-time augmentation reaches 77.87% top-1 accuracy. Part B implements six network modules from scratch in NumPy and verifies them against PyTorch and finite differences; the assembled network trains to 42.3% validation accuracy, within 0.24 points of the PyTorch reference.

Part A: PyTorch Image Classification

A.1 Introduction

CIFAR-100 contains 60,000 colour images of size 32×32, distributed over 100 fine-grained classes and 20 superclasses [1]. The small spatial resolution makes the task a useful test of architectural choices: a network can easily memorize the training images while still lacking the capacity or inductive bias needed for good generalization. I therefore treated the work as a sequence of controlled comparisons rather than a search for one lucky configuration.

This course-work report first establishes a reference point with a four-layer 5×5 convolutional network. It then tests training stability and optimization choices, evaluates architectural changes including 3×3 stacks, spatial downsampling, width, depth, residual connections, depthwise-separable convolutions, and normalization, and finally retrains the selected configuration on the full training set. The result is interpreted through both aggregate accuracy and the structure of the remaining errors.

The controlled progression raises validation accuracy from 11.33% for the short-budget SGD baseline to 74.13% for the selected wide residual model at 60 epochs. After full-data retraining, the best three-model ensemble with horizontal-flip test-time augmentation reaches 77.87% top-1 accuracy and 94.36% top-5 accuracy.

A.2 Environment and Training Protocol

All models were implemented directly with PyTorch modules [6]. The experiments used Python 3.11.14, PyTorch 2.11.0 with CUDA 12.8, torchvision 0.26.0, and an NVIDIA RTX 4060 Laptop GPU with 8 GB memory. The controlled comparisons used float32 arithmetic. Automatic mixed precision (AMP) was measured separately as an engineering comparison and was not used to decide which model was better.

The CPU/GPU and AMP measurements are summarized in Table 1. The downsampled CNN reaches a 7.0× GPU speed-up over the CPU. For the 20.7M-parameter final architecture, AMP reduces the step time from 164 ms to 87 ms, a 1.89× speed-up, while peak allocated memory falls from 1,379 MB to 861 MB. These measurements explain why the longer experiments were feasible, but they do not change the accuracy comparisons.

The validation protocol used the same deterministic 45,000/5,000 train/validation split for every controlled experiment. Each reported multi-seed result averages seeds 0, 42, and 123; a single-seed screen is explicitly marked as such. A run that collapses to a constant predictor is kept and reported in the text, and is excluded from the reported mean and standard deviation. The batch size was 128. The loss was cross-entropy. Unless stated otherwise, the optimizer was Adam [3] with learning rate 10^{-3} and a two-epoch linear warmup followed

Table 1: Hardware and throughput measurements for a batch of 128.

Device or mode	Model	Time per step (ms)	Images/s
CPU, float32	Downsampled	185	691
GPU, float32	Downsampled	27	4,838
GPU, float32	Final model	164	780
GPU, AMP	Final model	87	1,471

by cosine decay following the cosine-scheduling idea of [5]. The final retraining used all 50,000 training images and kept the 10,000-image test set untouched until training had finished. The complete settings are collected in Table 2.

Table 2: Common training settings.

Setting	Controlled comparisons	Final retraining
Data split	45k train / 5k validation / 10k test	50k train / 10k test
Batch size	128	128
Optimizer	Adam, 10^{-3} (SGD only for the reference)	Adam, 10^{-3}
Schedule	2-epoch warmup + cosine decay	2-epoch warmup + cosine decay
Epochs	10, 20, or 60	60
Augmentation	none or crop + horizontal flip	crop + horizontal flip
Seeds	0, 42, 123	0, 42, 123

A.3 Dataset and Preprocessing

The input pipeline converts each image to a channel-first float tensor and normalizes it with the training-set statistics

$$\mu = (0.5071,\ 0.4866,\ 0.4409), \qquad \sigma = (0.2673,\ 0.2564,\ 0.2762).$$

The validation and test transforms contain only tensor conversion and normalization. The augmented training transform adds a random crop to 32×32 with four-pixel padding and a random horizontal flip. No random transform is applied during evaluation. Figure 1 shows denormalized training images with their labels; the grid was used to check channel order, image shape, and label alignment before training.



Figure 1: Representative CIFAR-100 training images after reversing normalization for display. Each image is a 32×32 RGB tensor in channel-first form before plotting.

A.4 Baseline

A.4.1 Architecture

The reference network keeps the spatial resolution at 32×32 through four convolutional layers:

$$\begin{aligned} &\mathrm{Conv}(3, 256, 5) \to \mathrm{ReLU} \to \mathrm{Conv}(256, 256, 5) \to \mathrm{ReLU} \\ &\quad \to \mathrm{Conv}(256, 256, 5) \to \mathrm{ReLU} \to \mathrm{Conv}(256, 256, 5) \to \mathrm{ReLU} \\ &\quad \to \mathrm{GAP} \to \mathrm{Linear}(256, 100). \end{aligned} \tag{1}$$

All convolutions use padding two. The network in Eq. 1 has 4,961,124 trainable parameters. This is a useful reference because it is intentionally simple: it has no normalization, no downsampling, and no skip connections. The main architecture families are summarized in Table 3.

Table 3: Architecture summary for the main model families.

Model	Stage widths	Strides	Structure	Parameters (M)
Reference	$256, 256, 256, 256$	$1, 1, 1, 1$	four 5×5 convolutions	4.96
Downsampled	$64, 128, 256, 256$	$1, 2, 2, 2$	two 3×3 blocks/stage	2.35
Selected	$128, 256, 512, 512$	$1, 2, 2, 2$	two residual blocks/stage	20.70

A.4.2 Ten-epoch result

The baseline was trained for exactly 10 epochs with SGD and learning rate 10^{-2}. The first row of Table 4 is the reference. The remaining rows are a staged sequence in which the previous configuration is changed by one main factor at a time. Values are validation top-1 and top-5 accuracy in percent; the uncertainty is the sample standard deviation across the seeds used for each row.

Table 4: Short-budget progression on the fixed 45k/5k validation split. Uncertainties are sample standard deviations over the seeds used for each row: three seeds, except for Baseline + Adam, whose mean and standard deviation use the two non-collapsed seeds (the third collapsed to 0.76% top-1 and 3.92% top-5).

Configuration	Augmentation	Top-1 (%)	Top-5 (%)
Baseline + SGD	none	11.33 ± 0.30	31.95 ± 0.23
Baseline + Adam	none	31.23 ± 4.65	61.20 ± 4.89
Add BatchNorm	none	47.17 ± 0.25	76.99 ± 0.58
Add warmup + cosine decay	none	54.09 ± 0.11	82.77 ± 0.21
Add crop + flip	crop + flip	49.93 ± 0.18	79.99 ± 0.33
Replace 5×5 with stacked 3×3	crop + flip	54.25 ± 0.53	83.64 ± 0.23
Downsampled 3×3 CNN	crop + flip	61.00 ± 0.82	87.64 ± 0.24

On the official 10,000-image test set, the original full-data run of the same baseline (50,000 training images, seed 42) reaches 11.82% top-1. Table 4 instead uses the fixed 45k/5k split so that every controlled comparison shares one held-out set.

The baseline curves in Figure 2 show that the ten-epoch model is still under-trained: the final training top-1 accuracy is only about 10%. That training value is a running average accumulated while the weights are updated within the epoch, whereas validation is measured after the epoch with the final weights; the two curves are therefore not expected to coincide at the same epoch index. Replacing SGD with Adam raises the mean over the two non-collapsed seeds (27.94% and 34.52%) to 31.23 ± 4.65, but it also produces a severe seed-dependent failure: one of the three seeds collapses to 0.76% validation accuracy and is excluded from that mean. BatchNorm removes this collapse: the same seed reaches 47.04% instead of 0.76%, and the seed-to-seed standard deviation falls from 4.65 among the two healthy Adam runs to 0.25 over three BatchNorm seeds. The warmup–cosine schedule then gives the best short-budget result before architectural changes are introduced.

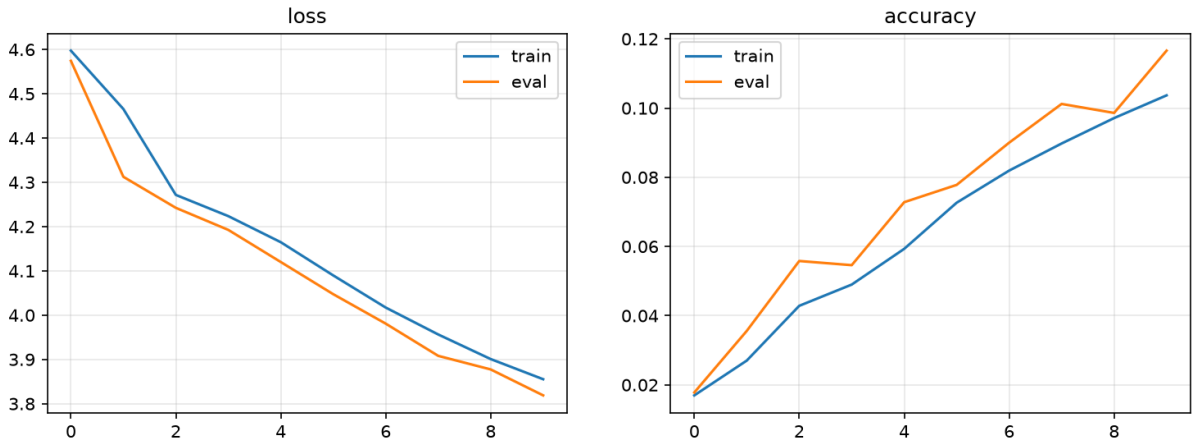


Figure 2: Training and validation curves for one baseline seed over ten epochs. The slow but steady improvement confirms that the reference network is not yet fitting the training set.

A.5 Experiments and Improvements

A.5.1 Experimental method

The main comparisons use three distinct training budgets. The 10-epoch block establishes the reference and exposes optimization failures. The 20-epoch block screens architectural factors under a common augmented recipe. The 60-epoch block confirms promising factors and measures the effect of longer training. Keeping these budgets separate avoids attributing a gain from additional optimization steps to an architectural change.

For each comparison I recorded the motivation, changed one primary factor, measured validation accuracy, and retained both positive and negative results. This is particularly important for changes whose effect is smaller than the approximately one-percentage-point noise floor observed between repeated data-loader runs.

The factor set was chosen for depth rather than breadth. Dropout and weight decay (AdamW) are listed as possible regularizers in the brief but were not swept here. The classifier is a single $128 \to 100$ linear layer after global average pooling, so dropout at that point has little capacity to remove; and a fair AdamW test requires its own decoupled weight-decay scale and learning-rate sweep, because the two interact. Muon, which is designed around matrix-shaped parameters, would need additional treatment for convolutional kernels. The experiments therefore concentrate on normalization, the schedule, augmentation, and architecture, where each change admits a clean single-factor comparison.

A.5.2 Normalization, optimization, and spatial structure

BatchNorm [2] is the first major stability intervention. Without it, the combination of a high-dimensional input stem and Adam can produce an almost constant predictor for one random seed. With BatchNorm, the same seed reaches 47.04% instead of 0.76%. The warmup phase limits the initial update size, while cosine decay provides a smooth reduction of the learning rate after the network has entered a useful region.

The stacked 3×3 design has the same nominal receptive field as a 5×5 layer because two successive 3×3 kernels cover a 5×5 neighbourhood. It also introduces an additional nonlinearity and uses fewer weights for equal channel counts. The downsampled model goes further: it begins with 64 channels and uses stages with widths $(64, 128, 256, 256)$ and strides $(1, 2, 2, 2)$. Reducing spatial resolution makes later convolutions substantially cheaper and leaves capacity for more useful feature transformations. In the 10-epoch block, this model reaches 61.00%, 6.75 points above the stacked 3×3 model trained with the same augmentation.

A.5.3 The role of augmentation and training duration

Crop-and-flip augmentation is not uniformly beneficial at a short budget. At ten epochs it lowers the BatchNorm baseline from 54.09% to 49.93%. This is consistent with stronger regularization slowing memorization before the model has enough optimization time. The sign changes with a longer budget, as shown in Table 5.

The long-run curves show that the non-augmented wide models reach almost 100% training accuracy early and then flatten or decline on validation data. Augmentation delays this memorization and continues to improve

Table 5: Effect of the same augmentation under a 60-epoch budget. All entries use three seeds and the 45k/5k validation split; uncertainties are sample standard deviations.

Architecture	No augmentation	Crop + flip	Difference
Wide ×2	61.47 ± 0.25	72.48 ± 0.40	+11.01
Deep ×2 + residual	58.13 ± 0.06	71.46 ± 0.25	+13.33

validation accuracy through the end of the 60-epoch schedule. Thus, the short-budget negative result does not contradict the long-budget positive result; the two measurements probe different stages of the optimization process.

A.5.4 Architecture ablations

The 20-epoch architecture screen is summarized in Table 6. Every entry uses Adam, warmup–cosine decay, crop-and-flip augmentation, and batch size 128. The anchor is the downsampled CNN. Several changes are intentionally negative: depth alone, depthwise-separable convolution, and GroupNorm all reduce accuracy in this setting. Width is the strongest single variable, while depth becomes useful only after residual connections in the spirit of [4] make the deeper optimization problem easier. The very small uncertainty on the width row is genuine: the three seeds give 70.38, 70.40, and 70.42.

Table 6: Architecture screen at 20 epochs. The ± values are sample standard deviations over three seeds; single-seed screens are marked by $n = 1$. The selected width and depth-plus-residual configurations were rechecked with three seeds.

Change relative to the downsampled anchor	Validation top-1 (%)	Seeds
Downsampled anchor	67.63 ± 0.98	$n = 3$
Residual connections only	67.40	$n = 1$
Width ×2	70.40 ± 0.02	$n = 3$
Depth ×2 only	64.52	$n = 1$
Depth ×2 + residual connections	69.15 ± 0.61	$n = 3$
Remove the 4×4 stage	63.12	$n = 1$
Depthwise-separable convolutions	54.58	$n = 1$
Depthwise + compensating width	65.00	$n = 1$
GroupNorm instead of BatchNorm	53.92	$n = 1$

The capacity comparison was then extended to 60 epochs. The final candidate combines width ×2, depth ×2, and residual connections. Its 20.7M parameters are spent on ordinary 3×3 convolutions rather than depthwise factorization. The resulting validation accuracy is 74.13%, higher than either width-only or depth-only scaling, as summarized in Table 7. The first visualization below isolates the gain from each architectural factor and the change in the augmentation response.

Table 7: Long-budget architecture comparisons at 60 epochs. All entries use crop-and-flip augmentation and three seeds; uncertainties are sample standard deviations.

Model	Parameters (M)	Validation top-1 (%)
Downsampled anchor	2.35	69.40 ± 0.38
Wide ×2	9.35	72.48 ± 0.40
Deep ×2 + residual	5.19	71.46 ± 0.25
Deep ×3 + residual	7.93	71.98 ± 0.13
Deep ×4 + residual	10.66	71.99 ± 0.25
Wide ×3	21.00	73.40 ± 0.50
Wide ×2 + deep ×2 + residual	20.70	74.13 ± 0.19
Wide ×2 + Cutout	9.35	73.09 ± 0.47

Table 7 shows the same pattern numerically: widening improves the score more consistently than adding residual depth, and the combined configuration gives the highest validation accuracy. Figure 3 makes the incremental gains and their seed variability easier to compare.

The capacity plot in Figure 4 complements Figure 3: width continues to improve the validation score over the tested range, whereas extra residual depth saturates after the third block per stage. This visual comparison is

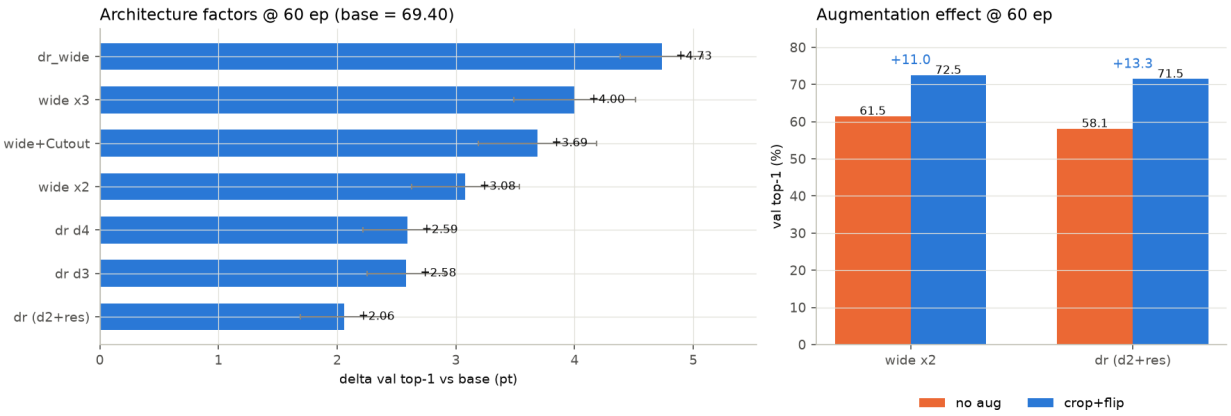


Figure 3: Validation gains at 60 epochs. The left panel compares architectural changes with the 2.35M-parameter anchor; the right panel isolates the augmentation effect for two architectures. Error bars show seed variability.

why the final model combines moderate depth with a wider channel schedule rather than using the deepest candidate.

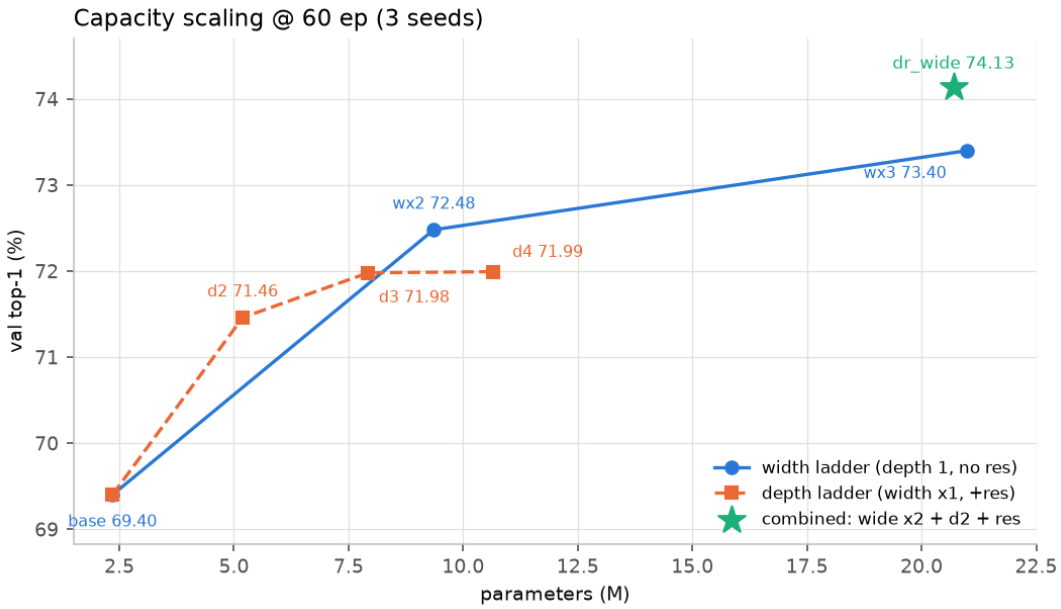


Figure 4: Width and depth capacity ladders at 60 epochs. Width keeps helping at the tested scales, whereas depth saturates after three residual blocks per stage. The star is the combined final candidate.

A.5.5 Selected model and training curve

The selected architecture has four stages with widths $(128, 256, 512, 512)$ and strides $(1, 2, 2, 2)$. Each stage contains two residual blocks, and each block contains two 3×3 convolutions with BatchNorm and ReLU. A 1×1 projection is used in a shortcut whenever the spatial size or channel count changes. Global average pooling and a 100-way linear classifier produce the logits. This design has 20,702,308 trainable parameters.

Figure 5 displays one 60-epoch validation run. The raw validation accuracy rises from 22.76% to 74.20%, while the EMA curve provides a smoother mid-training estimate. The raw and EMA curves converge near the end of cosine decay, so the final comparison uses the raw model weights.

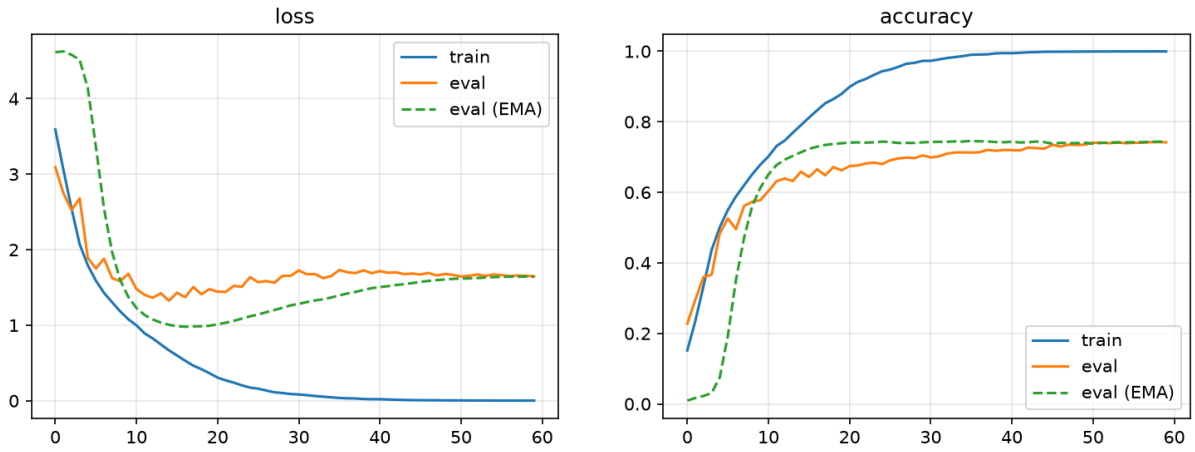


Figure 5: Training and validation curves for one seed of the selected model on the 45k/5k protocol. The dashed curve is an exponential moving average used for monitoring.

A.6 Final Model and Error Analysis

After model selection on the validation protocol, the selected architecture was retrained for 60 epochs on all 50,000 training images for seeds 0, 42, and 123. The test set was evaluated after training. Test-time augmentation (TTA) averages the predictions for the original image and its horizontal flip. The three-seed ensemble averages class probabilities before taking the prediction. The resulting test metrics are listed in Table 8.

Table 8: Final test-set results after full-data retraining. Uncertainties are sample standard deviations over three seeds.

Evaluation	Top-1 (%)	Top-5 (%)
Single model, mean ± std	75.01 ± 0.39	93.24 ± 0.03
Single model + TTA, mean ± std	75.69 ± 0.40	93.48 ± 0.13
Three-model ensemble	77.57	94.12
Three-model ensemble + TTA	**77.87**	**94.36**

The single-model result is already 0.88 points above the 45k/5k validation estimate, while probability averaging contributes a further 2.56 points over the single-model mean. TTA adds 0.30 points to the ensemble top-1 score. All four evaluation modes are reported because they measure different deployment choices.

A.6.1 Superclass-level confusion

For interpretable error analysis, I aggregated the 100 fine classes into the 20 CIFAR-100 superclasses using the dataset metadata. The confusion matrix in Figure 6 is computed from the three-model ensemble without TTA, whose top-1 accuracy is 77.57%. It contains 2,243 errors, of which 812 (36.2%) remain inside the correct superclass. Among wrong predictions, a uniform classifier would place only about $4/99 \approx 4\%$ in the correct superclass, so the errors are strongly concentrated among semantically related categories.

The most frequent confusions include boy → man (18), oak tree → maple tree (16), girl → woman (16), otter → seal (13), and maple tree → oak tree (13). The diagonal structure at the superclass level indicates that the model has learned useful coarse semantics even when fine-grained visual distinctions remain difficult.

A.6.2 Per-class accuracy and representative errors

Figure 7 sorts the 100 fine classes by accuracy. The weakest classes are boy (48%), girl (53%), otter (54%), bowl (57%), and seal/possum (59%). The strongest classes are tractor (97%), motorcycle (96%), and plain, wardrobe, and orange (94%). The gap is consistent with the confusion matrix: object categories with distinctive shapes are easier than small, low-resolution classes that share colour and texture with nearby categories.

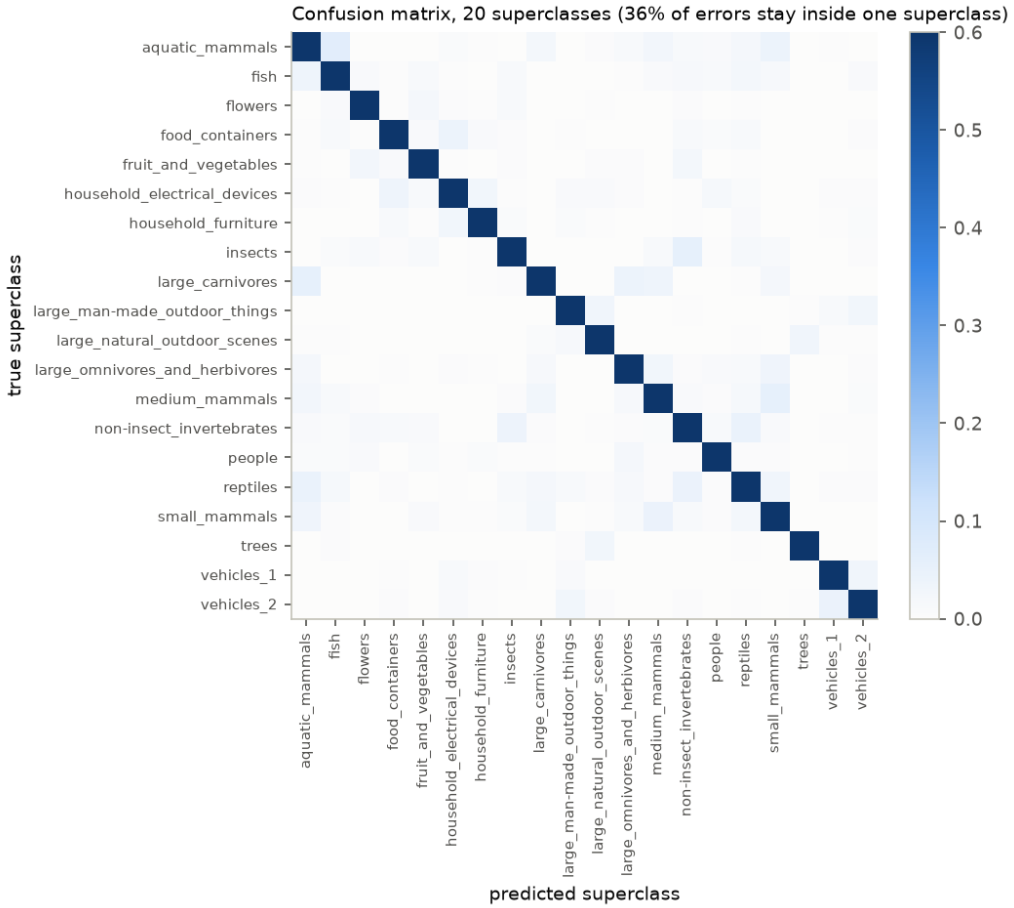


Figure 6: Row-normalized confusion matrix aggregated over the 20 superclasses. The diagonal is strong, while the remaining mass is concentrated in semantically close groups.

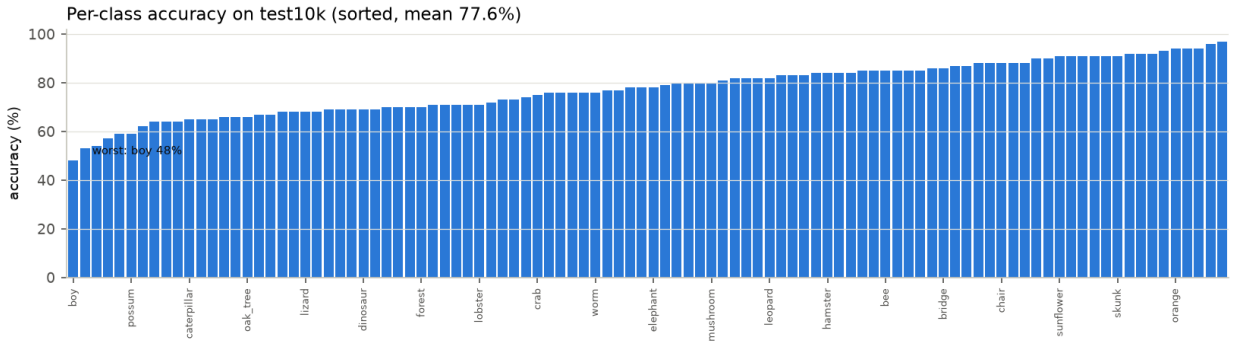


Figure 7: Fine-class accuracy for the three-model ensemble, sorted from lowest to highest. The mean is 77.6%.

Figure 8 complements the class-level plot with high-confidence mistakes. Several errors are visually understandable at 32×32: a lobster is predicted as a crab, a bicycle as a motorcycle, and a cup as a bottle. The model is therefore often confident about a plausible superclass or object family, but its fine label is not always recoverable from the pixels.

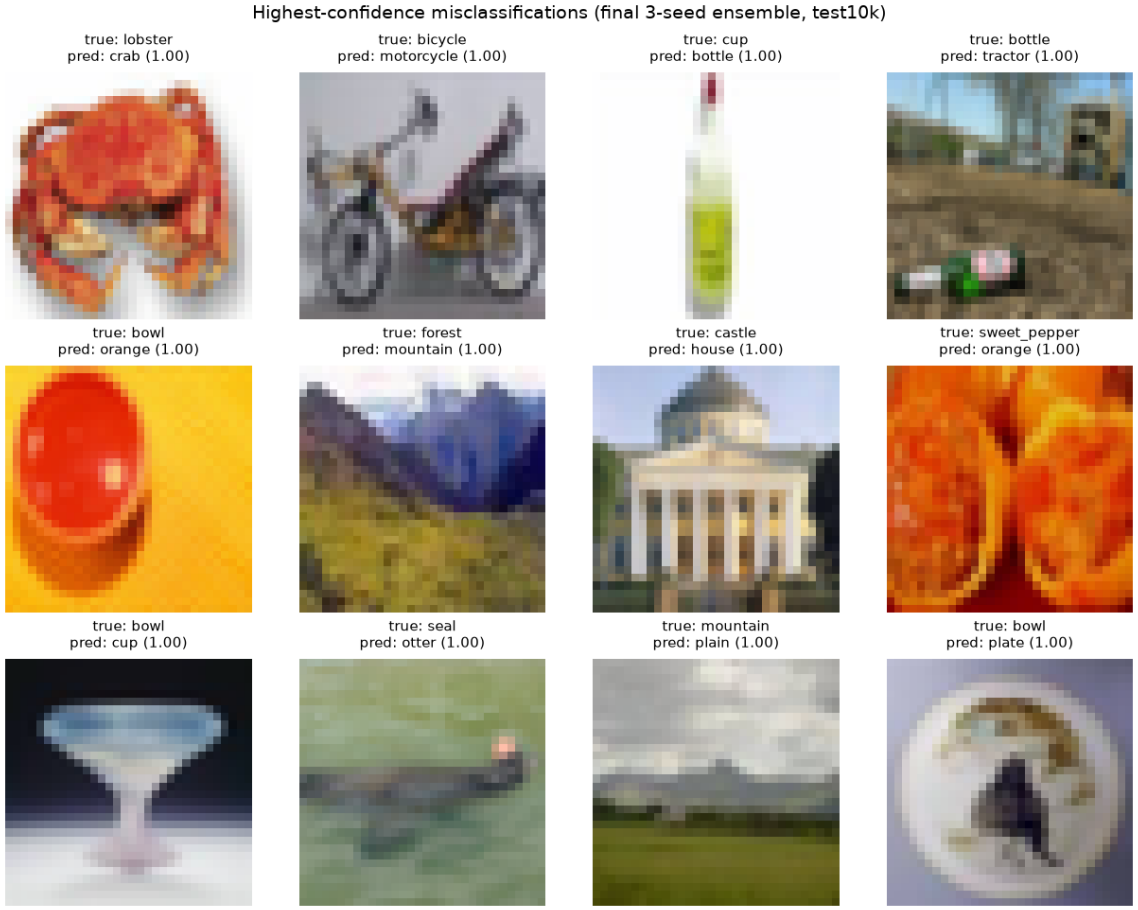


Figure 8: Examples of high-confidence errors from the final ensemble. Each title gives the true class, predicted class, and confidence.

A.7 What I Learned from Part A

1. **Stability can dominate optimizer choice.** Adam alone improved the mean over SGD, but one of three seeds collapsed to 0.76%; excluding that seed, the two healthy runs average 31.23%. BatchNorm changed that unstable regime into a repeatable 47.17% result. A mean reported without a stability check can therefore be misleading.
2. **The value of augmentation depends on the training budget.** Crop-and-flip was 4.16 points worse at ten epochs, yet 11.01 and 13.33 points better for the wide and deep-residual models at 60 epochs. The early loss is a regularization delay, not evidence that the transform is intrinsically harmful.
3. **Capacity should be spent where the data can use it.** Width produced the strongest single architectural gain. Depth alone reduced accuracy by 3.11 points at 20 epochs, but adding residual connections recovered the loss and produced a positive 1.52-point change in the replicated comparison. The residual path made additional depth trainable rather than automatically useful.
4. **Small improvements need replication.** A one-point difference appeared between repeated runs with the same nominal configuration because worker-side augmentation randomness was not controlled by the main-process seed. This made the three-seed confirmations more informative than a single best run and prevented treating sub-point changes as firm conclusions.
5. **Error structure is informative.** More than one third of the final errors stay inside one superclass, far above the random reference. The remaining problem is therefore fine-grained discrimination among related categories, which suggests that better local features or higher-resolution information would be more promising than another blind increase in depth.

Part B: Neural Network from Scratch

This part implements the forward and backward propagation of six network modules with NumPy only, then compares each module and the assembled network with independent PyTorch references. The modules are checked by at least two independent paths, and the controlled whole-network parity test reaches machine-precision agreement.

B.1 Overview and Scope

Implementation. The package `npnn` provides `Linear`, `GELU` (tanh approximation), `CrossEntropyLoss`, `Adam`, `Conv2d` (including im2col and col2im) and `SwitchNorm2d`, each with an explicit `forward` and `backward`. Each module is a Python class. The constructors expose the requested configuration fields: feature dimensions and bias for `Linear`, learning rate for `Adam`, channel counts, kernel, stride and bias for `Conv2d`, and the feature count for `SwitchNorm2d`. All numerical operations use NumPy; PyTorch is used only as a reference and is absent from the package.

Data protocol. The CIFAR-100 Python pickles are read directly with `pickle`; no framework data loader is used. Images remain `uint8` in channel-first layout until a batch is formed, then are converted to float32 and normalized with $(x/255 - \mu)/\sigma$. The training set is split into 45k training and 5k validation images, while the 10k test set remains untouched.

Environment. The implementation used Python 3.11.14 with NumPy 2.4.6 (OpenBLAS). PyTorch 2.11.0 was used only as the reference; the PyTorch counterpart ran on CPU only. BLAS threads were fixed at four; Section B.4 shows why more threads made the bandwidth-bound parts of the network slower, not faster.

B.2 Verification Strategy

Three checks were used to make the implementation testable.

Static and dynamic purity guard. A guard script parses every file in `npnn/` with Python's `ast` module and asserts that all imports belong to {numpy ∪ standard library ∪ project-local modules}. The policy is *default-deny*: an unknown package is a violation even if it is not on a blacklist. The scan also follows project-local imports transitively, so a helper module cannot smuggle a framework in. It also inspects dynamic import calls (`importlib.import_module`). A second, dynamic layer imports `npnn` in a clean subprocess and runs a forward pass, a backward pass and one parameter update. It then asserts that no banned top-level module entered `sys.modules`. The subprocess is essential: the test suite itself imports PyTorch, so an in-process check would be meaningless. The guard is itself validated by a negative control that builds a fake package containing `import torch` and confirms that both layers reject it.

An independent ruler before the parts. A central-difference checker was written and validated before any module: on $f(x) = \sum x^3$ the analytic gradient $3x^2$ agrees to 1.0×10^{-10}, and a deliberately wrong gradient is rejected. Sweeping the step size from 10^{-2} to 10^{-8} reproduces the classical U-curve, with truncation error falling as h^2, rounding error rising as h decreases, and a minimum near $h = 10^{-5}$; the 10^{-6} relative-error threshold used throughout follows from this measurement rather than from convention.

Two independent paths per module. Each differentiable module is checked against PyTorch, either through a corresponding layer or an independent tensor implementation of the same equation, and against central differences that share no code with the analytic backward pass. Adam has no input gradient, so it is checked instead by a 100-step PyTorch trajectory, a scalar hand calculation, and a checkpoint round trip. All numerical-gradient checks run in float64; float32 is rejected because rounding noise would obscure the $h = 10^{-5}$ difference.

B.3 Module Implementations

Throughout, N is the batch size and C the number of channels. Relative errors use $|g_{\text{num}} - g_{\text{ana}}| / \max(1, |g_{\text{num}}| + |g_{\text{ana}}|)$.

B.3.1 GELU

The implementation uses the tanh approximation

$$\mathrm{GELU}(x) \approx 0.5x\,(1+\tanh u)\,, \qquad u = \sqrt{\tfrac{2}{\pi}}\left(x + 0.044715x^3\right), \tag{2}$$

with derivative

$$\frac{d}{dx} = 0.5(1+\tanh u) + 0.5x\left(1-\tanh^2 u\right)\sqrt{\tfrac{2}{\pi}}\left(1+3\cdot 0.044715x^2\right). \tag{3}$$

The backward pass uses $1-\tanh^2 u$ rather than cosh, which would overflow for large $|u|$. The constant $\sqrt{2/\pi}$ is stored as a Python float: a NumPy scalar would silently promote float32 activations to float64. Compared with the exact erf version, the approximation differs by at most 4.732×10^{-4} at $x=-2.70$ (relative 5.0%), which matches the value predicted for this activation; the comparison must use `approximate='tanh'`, since PyTorch's default GELU is the erf variant.

B.3.2 Linear

For $X\in\mathbb{R}^{N\times d_{\text{in}}}$, $W\in\mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ and bias b, the forward pass is $Y=XW^\top+b$ and $dX=dY\,W$, $dW=dY^\top X$, $db=\sum_{\text{batch}} dY$. The leading dimensions are flattened before the two matrix products; without this the outer-product sum that produces dW is wrong for three-dimensional inputs, a failure mode that only appears when such an input is tested. When `bias=False` the parameter table genuinely contains no bias, which is implemented by making the parameter list a live view over the layer's attributes rather than a separate dictionary. Initialisation reproduces `kaiming_uniform` with $a=\sqrt{5}$: the gain is $\sqrt{2/(1+a^2)}=1/\sqrt{3}$, hence bound $=\sqrt{3}\cdot\mathrm{gain}/\sqrt{d_{\text{in}}}=1/\sqrt{d_{\text{in}}}$.

B.3.3 Cross-Entropy Loss

With logits z and integer labels y,

$$\mathcal{L} = \frac{1}{N}\sum_n \left[\log\sum_c e^{z_{n,c}} - z_{n,y_n}\right], \qquad \frac{\partial\mathcal{L}}{\partial z} = \frac{1}{N}\left(\mathrm{softmax}(z) - \mathrm{onehot}(y)\right). \tag{4}$$

The max subtraction is performed before exponentiation. The effect is visible: on logits of magnitude 10^3 the stable implementation returns a finite loss while the same formula without the subtraction returns `inf`. The reduction is the mean, matching the PyTorch reference. A hand-computed 2×3 example reproduces the loss 0.753109 exactly and the gradient to 5.6×10^{-17}.

B.3.4 Adam

The update follows PyTorch's defaults ($\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon=10^{-8}$):

$$m_t=\beta_1 m_{t-1}+(1-\beta_1)g_t, \quad v_t=\beta_2 v_{t-1}+(1-\beta_2)g_t^2, \quad \theta_t=\theta_{t-1}-\eta\,\frac{m_t/(1-\beta_1^t)}{\sqrt{v_t/(1-\beta_2^t)}+\varepsilon}. \tag{5}$$

The step counter starts at one, so the first update is approximately $\eta\,\mathrm{sign}(g)$ regardless of gradient magnitude; starting at zero divides by zero. Epsilon sits *outside* the square root. Parameters without gradients are skipped rather than represented by zero arrays, so a `bias=False` layer trains correctly. The strongest evidence is a step-by-step trajectory comparison: starting from identical parameters and feeding identical gradients, the two implementations differ by at most 2.78×10^{-17} over 100 steps. Saving and restoring (m, v, t) reproduces an uninterrupted run bit-for-bit, whereas restoring only the parameters would restart the optimiser hot.

B.3.5 Conv2d

The layer defines $Y_{n,o,i,j}=b_o+\sum_{c,u,v} W_{o,c,u,v}X_{n,c,iS+u,jS+v}$ with $H_{\text{out}}=\lfloor(H+2P-K)/S\rfloor+1$ and parameter count $C_{\text{in}}C_{\text{out}}K^2+\mathbf{1}_{\text{bias}}C_{\text{out}}$, asserted at construction. The required constructor exposes no padding argument, so the layer performs no padding; an optional padding argument is retained only for boundary tests.

Two implementations are kept. A naive version loops over output positions and contracts each patch with the filter; it is slow but almost impossible to get wrong, and serves as the oracle. The training path uses

Table 9: Verification summary. Forward errors are maximum absolute differences against PyTorch in float64; backward errors are maximum relative errors over all parameter gradients; the last column records either a central-difference check or the independent optimizer trajectory check.

Module	Cases	Forward max $\lvert\Delta\rvert$	Backward max rel. err.	Gradient or trajectory check
Linear	4	2.2×10^{-16}	3.4×10^{-16}	7.9×10^{-11}
GELU	3	8.9×10^{-16}	3.7×10^{-15}	1.6×10^{-10}
CrossEntropyLoss	3	0	3.5×10^{-18}	3.2×10^{-9}
Adam	4	—	2.8×10^{-17} (trajectory)	hand-checked
Conv2d	8	8.9×10^{-16}	3.8×10^{-15}	2.9×10^{-11}
SwitchNorm2d	3	8.9×10^{-16}	1.2×10^{-15}	1.3×10^{-10}

an explicitly written im2col + GEMM implementation rather than a pre-existing convolution operator: all patches are unfolded into rows of an $(NH_{\text{out}}W_{\text{out}}) \times (C_{\text{in}}K^2)$ matrix, so the convolution becomes one matrix product. The backward pass is the delicate part. dW is a second matrix product, but dX requires scattering every window's contribution back to the input, and overlapping windows mean that several contributions land on the same pixel. The implementation accumulates with K^2 strided slice additions rather than assigning, which is both correct and considerably faster than `np.add.at`.

An overlap bug is silent: with $S \geq K$ the windows do not overlap and an assignment-based implementation also passes, which is why the test matrix includes $K = 3, S = 1$. Eight configurations were verified (overlap, stride two, pointwise, $K = 5$, non-divisible height, $S \geq K$, no bias, padding), with forward agreement at 8.9×10^{-16}, parameter gradients at 3.8×10^{-15}, and a finite-difference check at 2.9×10^{-11}. The naive and vectorised implementations agree within numerical tolerance, while the vectorised forward is about 13× faster in the measured configuration.

B.3.6 SwitchNorm2d

Following Switchable Normalization [7], three statistics are computed for the same feature map: batch normalisation over (N, H, W), instance normalisation over (H, W) and layer normalisation over (C, H, W), with broadcasting shapes $(1, C, 1, 1)$, $(N, C, 1, 1)$ and $(N, 1, 1, 1)$ respectively. Their means and *biased* variances are combined with learnable weights $w = \text{softmax}(\lambda)$:

$$\hat{\mu} = \sum_k w_k \mu_k, \qquad \hat{\sigma}^2 = \sum_k w_k \sigma_k^2, \qquad y = \gamma \, \frac{x - \hat{\mu}}{\sqrt{\hat{\sigma}^2 + \varepsilon}} + \beta. \tag{6}$$

The normalisation uses the biased variance, matching PyTorch's batch-norm training mode; epsilon is added to the variance inside the square root. During training the mixed batch statistics are used and running averages of the mixed statistics are maintained for evaluation.

The discriminative test sets λ to a near one-hot vector, which makes the layer reduce to batch norm, instance norm or layer norm to numerical precision. All three degenerate cases agree with the corresponding PyTorch module to 8.9×10^{-16}, which simultaneously validates the reduction axes, the broadcasting, the biased-variance convention and the position of epsilon. The gradient with respect to x is obtained by differentiating through the mixed mean and variance and then distributing each branch back over its own reduction set; the gradient with respect to λ passes through the softmax Jacobian $\text{diag}(w) - ww^\top$. All four gradients $(x, \gamma, \beta, \lambda)$ agree with an independent PyTorch implementation to 1.2×10^{-15}, and the finite-difference check returns 1.3×10^{-10}.

B.3.7 Module Verification Summary

Table 9 collects the results. The measured errors are below 10^{-9} for forward comparisons and 10^{-6} for backward or finite-difference checks.

B.4 End-to-End Training and Comparison

B.4.1 Architecture and Budget

The network has three convolutional stages, each followed by SwitchNorm2d and GELU: $3 \to 32$ at stride one, $32 \to 64$ at stride two and $64 \to 128$ at stride two, then global average pooling and a linear layer to 100

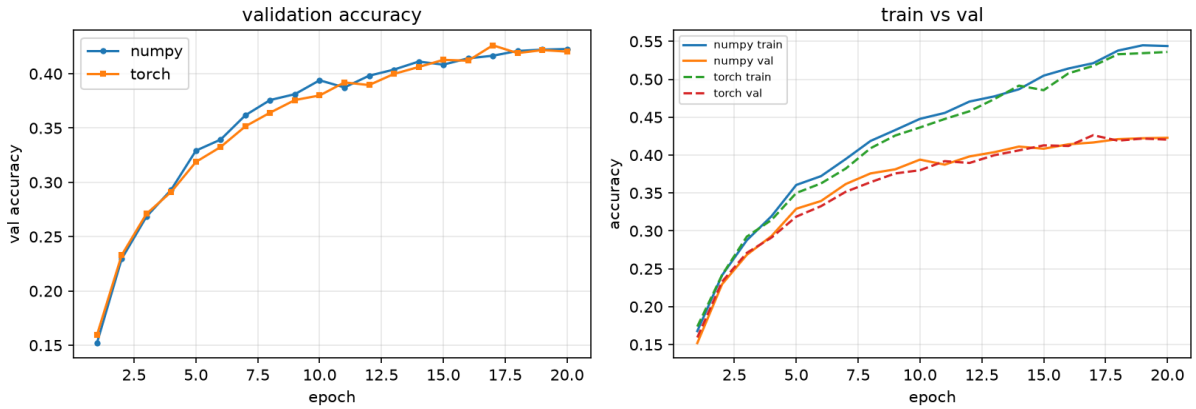


Figure 9: Left: validation accuracy of the NumPy implementation and the PyTorch reference under an identical budget. Right: training and validation accuracy for both. The curves remain close; the displayed gap includes the independent initializations and the normalization-state choices.

classes, for 107,268 parameters. The count is 896 for the first convolution, 18,496 and 73,856 for the next two, 12,900 for the classifier, and 160, 320 and 640 for the three SwitchNorm2d layers; because the switch logits λ are stored per channel, each normalization layer contributes $5C$ parameters (γ, β and $3C$ values of λ). The design uses early downsampling and 3×3 filters rather than a wide, full-resolution first layer, keeping the hand-written network small enough to train on the CPU.

Before training, a timing baseline measured matrix-multiplication throughput against thread count. Throughput rises from 98 to 328 GFLOP/s between one and four threads and then saturates, while the bandwidth-bound parts of the network (element-wise activations and the reductions inside SwitchNorm2d) became slower at eight and sixteen threads. Four threads were therefore fixed, which also leaves the machine usable for other work. The baseline deliberately separated compute from memory traffic, and it proved to be a *lower bound* for the convolutional layers: it measured the matrix product and a contiguous copy, but not im2col's strided gather or the output transpose. Profiling the assembled network located a larger hotspot still: NumPy routes the integer power x^3 through the generic `pow` kernel, which cost 57 ms per GELU forward pass on a $128 \times 32 \times 30 \times 30$ tensor. Replacing it with two multiplications reduced the epoch time from 320 s to 255 s, and the trained network remains bandwidth-bound rather than compute-bound.

B.4.2 Whole-Network Numerical Agreement

The strongest evidence in this part is not the final accuracy but the trajectory. The parity test uses a separate network from the 107,268-parameter training network: $3 \to 8$ and $8 \to 16$ convolutions, a flatten, and a $3136 \to 100$ linear layer, for 315,212 parameters (most of them in the classifier). Both implementations are generated from a single architecture specification, receive the same initial parameters (copied with shape assertions), the same float64 batches and the same Adam configuration, and run for 100 steps. The loss difference never exceeds 1.78×10^{-15}, and the largest parameter deviation at any step is 5.85×10^{-15} (5.76×10^{-15} at step 100). Since PyTorch has no switchable-normalisation layer, the reference computes the same equation with PyTorch tensors and autograd, so the agreement is between two independent implementations rather than between an implementation and itself.

B.4.3 Accuracy and Speed

The from-scratch network was trained on the full 45k training split for 20 epochs with Adam at 10^{-3} and batch size 128, reaching 54.38% on a 5,000-image training monitor and 42.28% validation accuracy. The PyTorch reference, trained with the identical architecture, data order, optimiser and budget, reaches 53.60% and 42.04% on the same monitor and validation sets. The validation curves track each other throughout training (Figure 9). The final difference is 0.24 percentage points; this supports similar training behaviour, while the independent initializations and normalization-state choices prevent a claim of identical trajectories.

Runtime differs substantially. Training alone took about 255 s per epoch in NumPy against 49.7 s in PyTorch, a factor of 5.1; including the two post-epoch evaluation passes, the full 20-epoch runs took 92 minutes against 18.6 minutes, a factor of 4.95. The gap is concentrated in the memory-bound operations rather than in the matrix products, where the two libraries call the same BLAS; it is the price of expressing im2col, col2im and

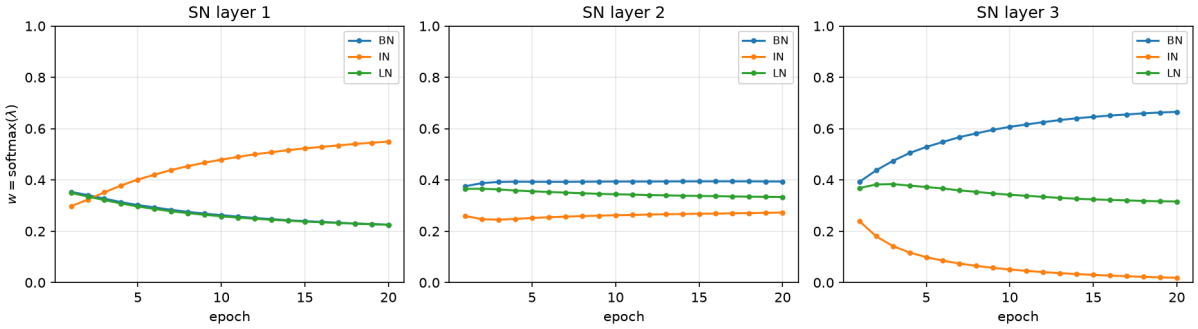


Figure 10: Learned switchable-normalisation weights $w = \mathrm{softmax}(\lambda)$, averaged over channels, for the three layers. The first layer moves towards instance normalisation, the third towards batch normalisation while instance normalisation collapses, and the middle layer stays close to uniform.

the three normalisation statistics as sequences of NumPy expressions instead of fused kernels. This is the concrete answer to the question of why a hand-written wheel is slower than an industrial one.

Figure 10 shows the depth-dependent behaviour described by the Switchable Normalization formulation [7]: the network does not choose one normalisation for all depths. The first layer increasingly prefers instance normalisation (final weight 0.55), which is consistent with low-level features needing per-image contrast normalisation, while the third layer converges to batch normalisation (0.67) and drives instance normalisation almost to zero. The middle layer remains close to uniform. This depth-dependent pattern is the strongest qualitative result of the part.

B.5 What I Learned from Part B

Silent bugs are the expensive ones. A convolution can appear to train while a wrong col2im update silently loses the contributions from overlapping windows; the error is exposed only by the $K = 3, S = 1$ case and an independent numerical-gradient check. Likewise, cross-entropy without max subtraction gives an infinite loss for logits of size 10^3. These checks showed me that a plausible training curve is not enough evidence for a low-level implementation.

A numerical ruler should come first. Writing and validating the central-difference checker before the six modules changed the debugging process. Once the checker had a known answer and a negative control, each later failure could be localized to one forward or backward equation instead of being diagnosed from end-to-end accuracy.

Numerical stability is part of the algorithm. Max subtraction in cross-entropy, $1 - \tanh^2(u)$ in GELU, the position of ε in normalization, and the use of biased variances all preserve the intended mathematics while changing the finite-precision behavior. The float64 checks made these choices observable rather than leaving them as implementation details.

Measurement corrected my intuition. A preliminary timing test measured only a matrix multiplication and a copy, so it underestimated the cost of a real convolution. Profiling the full network then showed that the cubic term in GELU was a major hotspot; replacing x^3 with two multiplications reduced the epoch time from roughly 320 seconds to roughly 255 seconds. The useful optimization came from measuring the implementation, not from guessing which line looked expensive.

A shared model definition makes a comparison meaningful. The NumPy and PyTorch training runs are generated from the same architecture specification, use the same split, data order, optimizer and 20-epoch budget, and reach 42.28% and 42.04% validation accuracy. The 0.24-point gap is small enough to support the conclusion that the manually implemented modules reproduce the reference training behaviour, while the independent initializations and normalization-state choices mean that it should not be read as a bit-for-bit training comparison.

Explicit scope is part of an honest result. The required convolution interface fixes padding at zero, with padding available only as a separately tested extension. SwitchNorm2d keeps running averages of its mixed statistics for inference, which is a practical approximation because instance and layer statistics depend on each input. Finally, the 20-epoch experiment answers whether the NumPy implementation can train the network and match a PyTorch counterpart; it is not a claim about the best possible CIFAR-100 accuracy. Recording these choices makes the result reproducible.

References

References

[1] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," technical report, University of Toronto, 2009.

[2] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of ICML*, 2015.

[3] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of ICLR*, 2015.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of CVPR*, 2016.

[5] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *Proceedings of ICLR*, 2017.

[6] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019.

[7] P. Luo, R. Zhang, J. Ren, Z. Peng, and J. Li, "Switchable Normalization for Learning-to-Normalize Deep Representation," arXiv:1907.10473, 2019.